DOCUMENTATION TECHNIQUE DE RÉFÉRENCE

Architecture complète du projet de quantification adaptative et de profilage matériel

Profiler V3/V5 — Quantification Algo — Z2/D2 Planner — Application Kimi K3

Auteur(s) : Équipe Projet

Date : 7 novembre 2026
Version : 1.0 — Architecture de référence
Statut : Document de conception vivant

Table des matières

1 Vision générale du projet 1
1.1 Les trois composants principaux 1
1.2 Philosophie 1
2 Profiler — Axes de profilage 2
2.1 Axe 1 — Cartographie structurelle du modèle 2
2.2 Axe 2 — Analyse des formats numériques 3
2.3 Axe 3 — Sensibilité à la quantification 3
2.4 Axe 4 — Allocation adaptative de précision 4
2.5 Axe 5 — Profilage mémoire 4
2.6 Axe 6 — Profilage I/O et transferts 5
2.7 Axe 7 — Profilage CPU / GPU / NPU 5
2.8 Axe 8 — Profilage spécifique MoE 5
2.9 Axe 9 — Profilage KV cache et contexte 6
2.10 Axe 10 — Profilage de performance et latence 6
2.11 Axe 11 — Profilage de qualité 6
2.12 Axe 12 — Prédiction des performances 7
3 Quantification Algo — Axes algorithmiques 8
3.1 Algorithme 1 — Allocation de précision mixte 8
3.2 Algorithme 2 — Quantification sensible aux activations 8
3.3 Algorithme 3 — Optimisation des échelles et des groupes 8
3.4 Algorithme 4 — Quantification hiérarchique 8
3.5 Algorithme 5 — Quantification MoE par expert 8
3.6 Algorithme 6 — Compression physique sans perte 8
3.7 Algorithme 7 — Optimisation du cache MoE 9
3.8 Algorithme 8 — Placement matériel adaptatif 9
3.9 Algorithme 9 — Optimisation du working set 9
3.10 Algorithme 10 — Recherche multiobjectif et Pareto 9
4 Z2/D2 Planner — Planification et décision 9
5 Validations expérimentales déjà réalisées 11
5.1 GTX 1080 — Qwen3.5-9B 11
5.2 GTX 1080 Ti — Bonsai-27B 11
5.3 MTP et décodage spéculatif 11
5.4 Qualcomm HTP et Adreno 11
6 Application au Kimi K3 12
6.1 Priorités de travail 12
6.2 Plateformes matérielles cibles pour la simulation 12
7 Architecture logicielle cible 12
8 Métriques de réussite du projet 13
9 Ordre de développement recommandé 14
10 Principes à ne pas perdre 14
11 Conclusion 14
12 Annexes 15
12.1 Structure recommandée du dépôt logiciel 15
12.2 Glossaire des abréviations 16

1 Vision générale du projet

L'objectif est de développer un **système complet** capable d'analyser, quantifier, profiler, planifier et exécuter des modèles de langage dont la taille dépasse largement la mémoire physique disponible.
Le principe fondamental est de **ne pas chercher uniquement à réduire le nombre de bits par poids**. Une approche qui se limiterait à compresser aveuglément les poids ignore les contraintes réelles d'exécution.
Il faut optimiser simultanément l'ensemble des dimensions suivantes :

- La précision numérique.
- La taille physique du modèle.
- La qualité de sortie.
- La mémoire résidente.
- Les transferts de données.
- La bande passante mémoire et stockage.
- La capacité de calcul des accélérateurs.
- La répartition CPU / GPU / NPU.
- Le coût de déquantification.
- Le comportement des modèles denses et MoE (Mixture of Experts).
- La latence et le débit réel en tokens par seconde.

> **Exigence clé**
>
> Le système doit être capable de prendre un modèle significativement plus grand que la mémoire disponible et de déterminer automatiquement **comment l'exécuter avec un working-set strictement limité**.

1.1 Les trois composants principaux

Quantification Algo	Recherche de représentations numériques optimales pour chaque tenseur
Profiler V3/V5	Analyse, mesure, prédiction et validation sur le matériel cible
Z2/D2 Planner	Décision, allocation des ressources et stratégie d'exécution

Le **runtime** constitue le quatrième élément de validation : il permet de confronter les prédictions du profiler aux performances réellement obtenues et d'alimenter une boucle de feedback d'adaptation.

1.2 Philosophie

Une quantification n'est pas bonne simplement parce qu'elle produit un fichier plus petit. Elle doit être évaluée selon son **coût total** :

$$C = f(Q, M, \mathrm{IO}, D, K, L)$$

avec :

- Q : qualité numérique préservée ;
- M : empreinte mémoire ;
- IO : coût des transferts (stockage $\rightarrow$ mémoire) ;
- D : coût de déquantification ;

- K : coût de calcul GEMM / kernels ;
- L : latence perçue par l'utilisateur.

> **Théorème informel du projet**
>
> Le meilleur format dépend du matériel, du tenseur, de la couche et du contexte d'exécution. Il n'existe pas de format universel optimal.

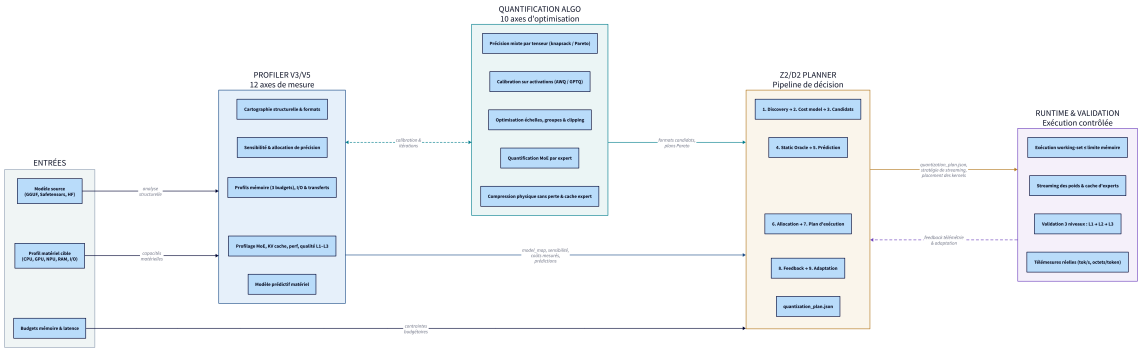


Fig. 1. – Vue d'ensemble de l'architecture : les quatre composants et leurs flux d'information.

2 Profiler — Axes de profilage

Le **Profiler V3/V5** est le système de mesure et de prédiction du projet. Il articule douze axes de profilage complémentaires.

2.1 Axe 1 — Cartographie structurelle du modèle

Objectif : connaître exactement la structure du modèle avant toute optimisation. Le profiler doit reconstruire l'organisation réelle des poids et des opérations.

Informations collectées :

- Nombre de couches ;
- Dimensions des tenseurs ;
- Nombre de paramètres ;
- Types de tenseurs ;
- Dimensions d'entrée et de sortie ;
- Architecture attentionnelle (MHA, GQA, MLA) ;
- Embeddings et LM Head ;
- MLP ;
- Normalisations (RMSNorm, LayerNorm) ;
- Attention dense ou latente ;
- Couches récurrentes ;
- Nombre d'experts et Top-K du routeur ;
- Organisation des blocs MoE.

Granularité : le système doit analyser quatre niveaux — (1) modèle complet, (2) couche, (3) tenseur, (4) famille de tenseurs. La **granularité expert** constitue une extension nécessaire pour les modèles MoE.

Sources : GGUF, Safetensors, checkpoints Hugging Face, métadonnées d'architecture, structures de tenseurs et formats propriétaires documentés.
Résultats produits : `model_map.json`, `layer_map.csv`, `tensor_map.csv`, `architecture_report.json`.
État : ✓ Implémenté analyse GGUF, couches, tenseurs, familles, dimensions et informations MoE structurelles. **À approfondir :** cartographie physique individuelle des experts et association avec leur identifiant réel.

2.2 Axe 2 — Analyse des formats numériques

Objectif : identifier ce que chaque format coûte réellement. Le profiler ne doit pas se limiter à lire l'étiquette Q4 ou Q8 : il doit comprendre la représentation physique.
Formats à prendre en charge : BF16, FP16, FP32, INT8, Q8_0, Q6, Q5, Q4, Q3, Q2, Q1_0, formats IQ, MXFP4, E8M0, ainsi que les formats expérimentaux propres au projet.
Analyse par format : pour chaque représentation sont quantifiés les bits par poids effectifs, la taille du bloc, le nombre de valeurs par bloc, la taille des échelles, les métadonnées, l'alignement mémoire, le padding, les dépendances du décodeur, la compatibilité CPU / GPU / NPU et le coût de déquantification.

> **Exemple : format Q1_0 (Bonsai-27B)**
>
> Le format Q1_0 étudié utilise des blocs de 128 éléments codés sur 18 octets, soit **1,125 bit par poids** effectif. Le profiler doit distinguer le coût théorique (1,125 bpw) du coût réel (overhead des échelles, temps de conversion).

Résultats : `quant_formats.py`, `format_capability_report.json`.
État : ✓ Implémenté registre de formats et analyse de plusieurs formats GGUF et formats de recherche.
Extension : intégrer les formats propres aux modèles MoE et les formats compressés sans perte.

2.3 Axe 3 — Sensibilité à la quantification

Objectif : découvrir quels poids peuvent être fortement quantifiés et lesquels doivent conserver une précision supérieure. Une quantification uniforme suppose que tous les poids ont une importance équivalente — hypothèse généralement fausse.
Mesures par couche et par tenseur :

- Distribution des poids (moyenne, écart-type, valeurs extrêmes) ;
- Erreur de reconstruction : RMS error, erreur maximale, **cosine similarity**, SNR ;
- Sensibilité aux changements de précision.

Méthodes : reconstruction par quantification, comparaison avec les poids originaux, analyse des distributions, calibration sur activations, comparaison de sorties et analyse de sensibilité par couche.
Fonction objectif de sensibilité :

$$E_i = \frac{\|W_i - \widehat{W}_i\|_2}{\|W_i\|_2 + \varepsilon}$$

Cette erreur n'est qu'un indicateur : elle ne remplace pas la mesure de l'impact sur les activations et les logits.
Résultats : `layer_sensitivity.csv`, `tensor_sensitivity.csv`.
État : ✓ Implémenté analyses de reconstruction et mesures RMS, maximum et cosine. **Extension :** calibration systématique par activation et sensibilité aux tâches.

2.4 Axe 4 — Allocation adaptative de précision

Objectif : sélectionner automatiquement la précision de chaque partie du modèle. C'est le cœur algorithmique de Quantification Algo.

Au lieu d'imposer un Q4 global, le système peut sélectionner :

- Q2 pour certains tenseurs peu sensibles ;
- Q3 pour d'autres ;
- Q4 pour les couches intermédiaires ;
- Q6 ou Q8 pour les couches sensibles ;
- BF16 pour les opérations qui doivent conserver leur précision intacte.

Variables de décision par tenseur i : taille originale S_i, taille quantifiée $S_{i,q}$, erreur $E_{i,q}$, coût de calcul $C_{i,q}$ et coût de transfert $T_{i,q}$. Le système doit respecter un budget mémoire :

$$\sum_i S_{i,q_i} \leq B$$

tout en minimisant la dégradation globale :

$$\min \sum_i w_i E_{i,q_i}$$

où w_i représente l'importance relative de la partie du modèle. Une version avancée intègre également le temps de calcul et de transfert dans l'objectif.

Algorithmes envisageables : allocation gloutonne, programmation dynamique, sac à dos multidimensionnel, optimisation sous contraintes, recherche locale, allocation basée sur Pareto, calibration adaptative.

Sortie : `quantization_plan.json` conservant chaque décision avec sa justification.

État : ✓ Implémenté planification de quantification par couche, tenseur et famille. **Extension :** optimisation multiobjectif intégrant simultanément qualité, résidence, bande passante et calcul.

2.5 Axe 5 — Profilage mémoire

Objectif : comprendre la différence entre taille du modèle et mémoire réellement nécessaire pour l'exécuter. Il faut séparer trois budgets distincts :

A. Storage budget	Taille totale du modèle sur le stockage (SSD / distant)
B. Memory budget	Mémoire disponible pour les poids, activations, caches et autres allocations
C. Movement budget	Quantité de données qu'il faut déplacer pendant l'exécution

Mesures : RAM totale et disponible, VRAM, working set, poids résidents, chargés et demandés, activations, KV cache, cache des experts, pic mémoire, fragmentation, page cache, mémoire DMA et VTCM.

Objectif concret : être capable d'exécuter un modèle supérieur à 8 GiB avec un working set inférieur ou égal à 8 GiB.

État : ✓ Implémenté estimations mémoire, résidence et plafonds de stockage dans V5. **Extension :** instrumentation plus précise de la mémoire physique, des allocations temporaires et du comportement page cache.

2.6 Axe 6 — Profilage I/O et transferts

Objectif : mesurer le prix réel de la lecture des poids — déterminant pour les modèles qui dépassent la RAM disponible.

Mesures : octets lus à froid / à chaud par token, nombre de requêtes, latence des lectures, débit SSD, cache hit/miss, lectures répétées, coût de décompression, temps de transfert, coût du mapping, DMA et RPC vers accélérateur.

Méthodes : lecture sélective de tenseurs, HTTP Range, streaming de poids, analyse des pages et comparaison des accès froids et chauds.

Métrique centrale :

$$\mathrm{IO}_{token} = \frac{\text{Octets effectivement lus}}{\text{Tokens générés}}$$

> **Règle impérative I/O**
>
> Il faut impérativement distinguer les octets demandés, les octets réellement lus depuis le stockage et les octets déjà disponibles dans le page cache.

État : ✓ Implémenté estimation **cold bytes/token**, résidence, plafonds et lecture HTTP Range de tenseurs. **Extension :** mesure live complète du stockage et attribution des transferts par couche et par expert.

2.7 Axe 7 — Profilage CPU / GPU / NPU

Objectif : déterminer où chaque opération doit réellement s'exécuter. Une opération théoriquement compatible avec un accélérateur n'est pas nécessairement exécutée sur celui-ci.

CPU : SIMD, AVX2, FMA, NEON, déquantification, GEMM, opérations scalaires.

GPU : CUDA, OpenCL, occupation, bande passante, temps kernel, transferts host/device, détection des fallbacks CPU.

Qualcomm HTP : QAIRT, HTP v81, VTCM, DMA, RPC, opérations supportées, placement réel des couches et fallbacks.

Adreno : OpenCL, kernels MUL_MAT, copies, temps d'exécution, compatibilité des formats.

Mesure indispensable :

$$T_{total} = T_{load} + T_{transfer} + T_{dequant} + T_{compute} + T_{sync}$$

> **Principe fondamental de placement**
>
> Ne jamais assimiler la compatibilité d'une opération à son accélération effective.

État : ✓ Implémenté base de capacités, analyse des formats, télémétrie Android, parsing HTP et OpenCL, analyses de copies et de placements. **Extension :** attribution systématique des coûts par opération et par tenseur.

2.8 Axe 8 — Profilage spécifique MoE

Objectif : passer d'une analyse de conteneurs d'experts à une véritable analyse de leur **comportement**. C'est l'axe à développer en priorité pour Kimi K3.

Informations structurelles : nombre total d'experts, dimensions, experts activés, Top-K, taille physique par expert, format numérique, organisation des poids et identifiants.

Informations dynamiques : expert sélectionné, fréquence de sélection, distribution de routage, réutilisation, cache hit/miss, évictions, temps de chargement, octets transférés, temps de calcul et impact sur la latence.
Indicateurs :

$$\text{HitRate} = \frac{\text{CacheHits}}{\text{CacheHits} + \text{CacheMisses}}$$

$$\text{ExpertReuse}(e) = \frac{N_e}{N_{total}}$$

Le profiler doit également mesurer l'entropie de distribution des experts : une distribution uniforme et une distribution concentrée n'impliquent pas du tout la même stratégie de cache.
État : ◖ Partiel informations structurelles MoE, nombre d'experts et Top-K présents. **Non implémenté de manière complète :** identifiants dynamiques, routage réel, cache par expert, évictions, résidence et quantification individuelle des experts.

2.9 Axe 9 — Profilage KV cache et contexte

Objectif : mesurer le coût de la mémoire de contexte — le poids du modèle n'est pas le seul facteur de consommation.
Mesures : tokens de contexte, taille KV par token, nombre de couches, de têtes, dimension des têtes, précision du KV (FP16, BF16, INT8, INT4), croissance mémoire, cache MLA, états récurrents, temps de lecture/écriture.
Indicateur :

$$M_{KV}(L) \approx L \times M_{KV/token}$$

Pour les architectures MLA ou récurrentes, le calcul doit être adapté à la structure réelle du modèle.
État : ✓ Implémenté déjà étudié dans les scénarios de capacité mémoire et de contextes longs. **Extension :** instrumentation dynamique homogène sur les différents backends.

2.10 Axe 10 — Profilage de performance et latence

Objectif : expliquer chaque seconde consommée pendant la génération.
Mesures : prefill tok/s, decode tok/s, temps par token, latence initiale, moyenne, P50, P95, P99, variance, temps de chargement, de déquantification, GEMM, transferts et synchronisations.

> **Règle de publication des débits**
>
> Ne jamais publier un seul débit sans préciser : le matériel, le backend, le format, le contexte, le nombre de répétitions, l'état du cache (froid ou chaud) et la méthode de mesure.

État : ✓ Implémenté mesures expérimentales et outils de profilage disponibles. **Extension :** corrélation temporelle complète par opération et par token.

2.11 Axe 11 — Profilage de qualité

Objectif : garantir que les économies de mémoire ne détruisent pas les capacités du modèle.
Métriques : perplexité, cross-entropy, KL divergence, RMS error, erreur maximale, **cosine similarity**, SNR, **logit agreement**, **token agreement**, qualité de génération et stabilité du routage.
Niveaux de validation :

- **L1 — Structure et reconstruction :** poids, dimensions, formats, déquantification, erreurs numériques.
- **L2 — Comportement des opérations :** sorties intermédiaires, activations, logits, opérations critiques.
- **L3 — Validation du modèle :** perplexité, générations, comportement global et performances.

> **Règle de validation**
>
> La réussite de L1 ne démontre pas la réussite de L3.

État : ✓ Implémenté mesures de reconstruction et de qualité, architecture L1–L3. **Extension :** protocole uniforme sur tous les formats et backends.

2.12 Axe 12 — Prédiction des performances

Objectif : prédire les performances avant de lancer une quantification ou une exécution coûteuse. Le profiler doit devenir un **modèle prédictif matériel**.

Entrées : architecture, formats, dimensions, quantification, RAM disponible, VRAM, bande passante, débit stockage, capacité CPU/GPU/NPU, cache, longueur du contexte.

Sorties : taille finale, working set, pic mémoire, octets lus par token, temps de déquantification, temps de calcul, débit estimé, latence, risque de saturation.

Modèle simplifié :

$$T_{token} = T_{\mathrm{IO}} + T_{compute} + T_{dequant} + T_{sync} + T_{other}$$

Les temps ne sont pas toujours simplement additifs : le modèle final doit tenir compte du **chevauchement** entre transferts et calculs.

État : ✓ Implémenté prédictions de capacité, de mémoire et de coûts. **Extension :** modèle calibré par backend, capable de prédire avec des intervalles d'incertitude.

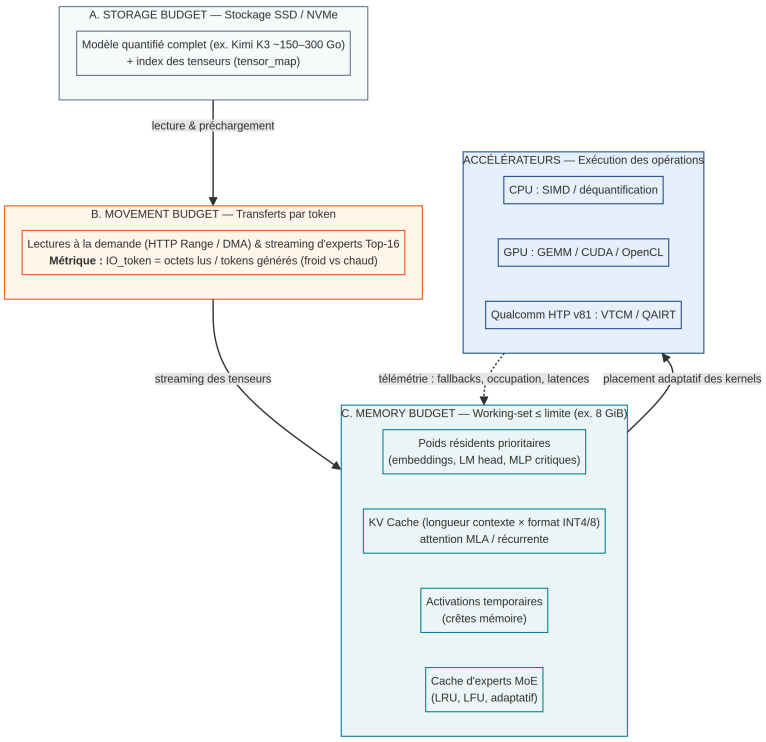


Fig. 2. – Modèle de mémoire à trois budgets et flux de streaming des poids et experts.

3 Quantification Algo — Axes algorithmiques

3.1 Algorithme 1 — Allocation de précision mixte

Le système choisit une précision **différente pour chaque couche et chaque tenseur** au lieu d'imposer un Q4 uniforme.

Fonctionnement en 10 étapes : (1) charger les poids source ; (2) identifier les tenseurs ; (3) calculer leur sensibilité ; (4) générer plusieurs candidats de quantification ; (5) mesurer leur erreur ; (6) calculer leur taille ; (7) estimer leur coût matériel ; (8) construire un plan ; (9) comparer les solutions Pareto ; (10) produire le checkpoint final.

Objectif : maximiser la qualité sous un budget mémoire et un budget de latence donnés.

3.2 Algorithme 2 — Quantification sensible aux activations

La quantification des poids doit être complétée par l'analyse des activations : deux tenseurs ayant une erreur de reconstruction similaire peuvent produire des effets très différents sur les sorties.

Méthodes candidates : calibration sur activations, GPTQ, AWQ, SmoothQuant, recherche de clipping, optimisation des échelles, **group-wise quantization**. Le système collecte des activations représentatives, teste les candidats et mesure l'effet sur les sorties, en respectant l'architecture et les contraintes de chaque format.

3.3 Algorithme 3 — Optimisation des échelles et des groupes

L'objectif est de choisir conjointement : la taille des groupes, les échelles, le zéro-point (lorsque le format le permet), le clipping, la distribution des valeurs et l'alignement des blocs. Le choix doit être évalué selon la précision et le coût d'exécution réel — une quantification à 4 bits avec des métadonnées trop lourdes peut être moins efficace qu'attendu.

3.4 Algorithme 4 — Quantification hiérarchique

Le système doit pouvoir optimiser successivement : (1) famille de tenseurs → (2) couche → (3) tenseur → (4) bloc → (5) expert → (6) éventuellement sous-structure d'expert.

Cette hiérarchie limite le coût de recherche : seules les parties sensibles sont affinées avec une granularité supérieure.

3.5 Algorithme 5 — Quantification MoE par expert

Extension particulièrement importante pour Kimi K3 : chaque expert peut recevoir sa propre allocation de précision.

Variables prises en compte : sensibilité de l'expert, fréquence d'activation, importance dans le routage, taille, coût de chargement et de calcul, budget mémoire.

Stratégie : un expert fréquemment utilisé et sensible conserve une précision élevée ; un expert peu utilisé et robuste reçoit une précision plus basse. **Attention :** la fréquence d'utilisation ne suffit pas, il faut également mesurer la sensibilité et l'impact sur les logits.

Résultat : `expert_quantization_plan.json`.

État : ◆ Axe de développement axe de développement, non validé dans V5.

3.6 Algorithme 6 — Compression physique sans perte

Il faut distinguer deux opérations fondamentalement différentes :

- **Modifier les valeurs numériques** par quantification (avec perte contrôlée) ;

• **Compresser la représentation physique** sans modifier les valeurs (sans perte).

Pour les experts MXFP4, cette distinction est essentielle. L'objectif est de réduire les lectures SSD et les transferts sans introduire d'erreur numérique supplémentaire, en décompressant au chargement ou pendant le streaming.

Mesures : ratio de compression, débit de décompression, CPU consommé, octets physiques lus vs octets logiques reconstruits, impact sur la latence.

3.7 Algorithme 7 — Optimisation du cache MoE

Le cache LRU classique n'est pas nécessairement optimal pour les experts MoE.

Stratégies à comparer : LRU, LFU, cache adaptatif, prédiction du prochain expert, préchargement, cache pondéré par taille, cache tenant compte du coût de chargement.

Objectif : minimiser $T_{cache} = T_{miss} + T_{load} + T_{eviction}$ tout en respectant le budget mémoire. La décision doit s'appuyer sur les **traces de routage réelles** et non sur une hypothèse de réutilisation.

3.8 Algorithme 8 — Placement matériel adaptatif

Le système doit sélectionner le backend approprié pour chaque opération : CPU vs GPU, CPU vs HTP, OpenCL vs CUDA, format directement supporté vs conversion, exécution locale vs transfert.

Critère clé : ne pas choisir le backend uniquement selon son débit théorique. Le coût complet inclut les transferts, la conversion, les synchronisations et les éventuels fallbacks — une opération plus rapide sur GPU peut être plus lente une fois le transfert pris en compte.

3.9 Algorithme 9 — Optimisation du working set

Le système construit un plan d'exécution compatible avec une limite mémoire imposée : quels poids restent résidents, quels poids sont chargés à la demande, quels experts sont préchargés, quelles couches peuvent être streamées, quelle quantité de KV cache peut être conservée et quelle réserve mémoire maintenir. Le but n'est pas seulement de faire tenir les poids, mais d'éviter que les transferts détruisent le débit.

3.10 Algorithme 10 — Recherche multiobjectif et Pareto

Le système doit éviter de rechercher une solution unique avant de connaître les compromis possibles. Les axes d'optimisation sont : qualité, taille, latence, débit, RAM, VRAM, I/O, énergie et compatibilité matérielle.

Une **solution Pareto** est une solution pour laquelle aucun objectif ne peut être amélioré sans dégrader au moins un autre objectif. Le planner sélectionne ensuite la solution la plus adaptée aux contraintes réelles de l'appareil cible.

4 Z2/D2 Planner — Planification et décision

Le planner exploite les résultats du profiler et de Quantification Algo. Il ne remplace pas le profiler : **il consomme ses observations et ses prédictions** pour construire des décisions reproductibles et justifiées.

1	Capability discovery	Identifier les ressources matérielles disponibles (CPU, GPU, NPU, RAM, débits)

2	Cost model	Calculer les coûts mémoire, I/O, calcul et transfert
3	Candidate generation	Générer plusieurs plans de quantification et d'exécution alternatifs
4	Static Oracle	Comparer les plans selon les informations connues avant exécution
5	Prediction	Estimer le débit, la latence et la qualité de chaque plan
6	Allocation	Répartir les poids, les caches et les opérations sur les ressources
7	Execution planning	Construire la stratégie d'exécution ordonnancée
8	Feedback	Comparer les performances prédites aux mesures réelles en cours d'exécution
9	Adaptation	Réviser les décisions lorsque les observations contredisent les prédictions

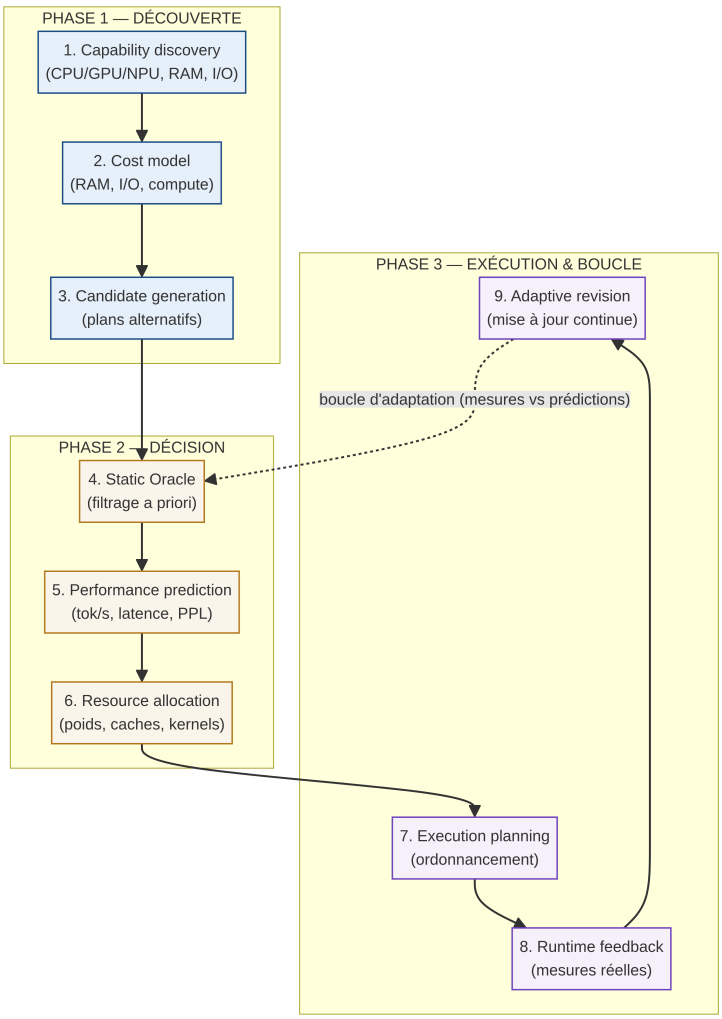


Fig. 3. – Le pipeline de décision en 9 étapes du Z2/D2 Planner avec boucle d'adaptation.

Le planner doit **conserver les décisions et leurs justifications** afin de garantir la reproductibilité et la traçabilité de chaque choix d'optimisation.

5 Validations expérimentales déjà réalisées

5.1 GTX 1080 — Qwen3.5-9B

Machine de test : NVIDIA GeForce GTX 1080 8 Go. Résultats documentés :

Taille GPU	5,388 GiB	5,079 GiB
Perplexité	9,7839	10,0437
Débit	33,42 tok/s	35,98 tok/s

Enseignement majeur : le format mixte améliore significativement la perplexité (+2,6 % de qualité) au prix d'une baisse très modérée de débit. Travaux associés : 442 tenseurs analysés, `tensor_map.csv`, analyse PPL, SNR, coûts par couche et par tenseur, variantes v7/v8/v4/v9/Q8_0. **Cette expérience constitue une validation concrète de l'intérêt de la précision non uniforme.**

5.2 GTX 1080 Ti — Bonsai-27B

Configuration ultra-compacte Q1_0 : 1,125 bit par poids, blocs de 128 éléments, 18 octets par bloc.

Modèle + contexte 4K	5,2 Go
Modèle + contexte 10K	5,6 Go
Modèle + contexte 100K (KV 4 bits)	6,8 Go
Débit rapporté	28,3 tok/s

Ces résultats démontrent l'intérêt d'étudier les représentations extrêmement compactes. **Ils ne doivent toutefois pas être confondus avec les performances attendues sur Kimi K3.**

5.3 MTP et décodage spéculatif

Résultats rapportés sur les expérimentations GTX 1080 :

- Environ 55–56 tok/s avec MTP ;
- Environ 61–61,3 tok/s avec une profondeur de 3 ;
- Taux d'acceptation autour de 0,702.

Cet axe doit rester distinct de la quantification : il agit sur le nombre de tokens acceptés par étape de décodage et **peut amplifier les gains** obtenus par une meilleure représentation des poids.

5.4 Qualcomm HTP et Adreno

Travaux réalisés sur les capacités Qualcomm : analyse QAIRT, HTP v81, analyse VTCM, parsing des placements, analyse des copies, OpenCL Adreno, analyse des opérations `MUL_MAT`, détection des fallbacks CPU.

Marco MoE — Adreno OpenCL	31,23 tok/s
Marco MoE — HTP	23,6 tok/s
Modèle 9B — Adreno OpenCL	10,19 tok/s

Modèle 9B — HTP	9,8 tok/s
Modèle 1,7B — Adreno OpenCL	38,3 – 38,95 tok/s
Modèle 1,7B — HTP	32,5 tok/s

Cas d'école : Bonsai-27B sur accélérateur mobile

Une expérimentation Bonsai-27B a montré un **fallback CPU de 99,7 %** sur les opérations MUL_MAT, avec seulement **0,32 tok/s mesuré contre 13,9 tok/s estimés initialement**. C'est un exemple essentiel de l'écart entre compatibilité théorique et accélération réelle.

6 Application au Kimi K3

Kimi K3 constitue un **nouveau cas d'utilisation** pour l'ensemble du système. Les dimensions clés à étudier sont : tronc dense BF16, experts MXFP4, routage Top-16, cache des experts, lecture à la demande, quantification mixte, décompression physique, prédiction du débit et allocation mémoire.

6.1 Priorités de travail

P1	Tronc dense BF16	Tester : BF16 de référence, INT8, INT4, mixte INT4/INT8, allocation adaptative par couche et par tenseur
P2	Experts MXFP4	Tester : conservation MXFP4, compression sans perte, quantification expérimentale supplémentaire, cache adaptatif, sensibilité par expert
P3	Simulation working set	Construire des simulations adaptées aux trois plateformes cibles matérielles

6.2 Plateformes matérielles cibles pour la simulation

1. **Station intermédiaire :** Ryzen 5 5600 · 32 Go RAM · NVIDIA GeForce RTX 3060 12 Go.
2. **Station moderne :** Ryzen AI 9 HX365 · 32 Go RAM · NVIDIA GeForce RTX 5070 8 Go.
3. **Mobile haut de gamme :** OnePlus 15 · Qualcomm HTP v81 · GPU Adreno.

Condition d'utilisabilité des accélérateurs

Les accélérateurs ne doivent être considérés comme utilisables que si leurs kernels et leurs chemins de transfert sont **réellement validés** sur le matériel concerné.

7 Architecture logicielle cible

Le système est découpé en quatorze modules modulaires. Les nouveaux noms sont des propositions d'organisation cible.

`capability_db.py`	Base de données des capacités matérielles (CPU, GPU, NPU, mémoires, débits)
`quant_formats.py`	Registre et propriétés physiques des formats numériques
`profile_model.py`	Cartographie structurelle des modèles (GGUF, Safetensors, HF)
`profiler.py`	Mesures mémoire, I/O et analyse de la résidence
`predict_from_hf.py`	Prédiction de performances depuis les modèles Hugging Face
`predict_from_hf_v5.py`	Prédiction avancée V5 avec plafonds de stockage et budgets
`parse_hexagon.py`	Analyse des traces d'exécution Qualcomm HTP (VTCM, fall-backs)
`quantization_engine.py`	Génération des candidats de quantification et reconstruction
`sensitivity_engine.py`	Calcul de la sensibilité numérique par couche et par tenseur
`mixed_precision.py`	Allocation optimale de précision sous contraintes mémoire
`expert_profiler.py`	Profilage dynamique MoE (experts, routage, fréquences)
`expert_cache.py`	Simulation et optimisation du cache d'experts (LRU, LFU, adap-tatif)
`cost_model.py`	Modèle de coûts d'exécution (IO + compute + dequant)
`planner.py`	Pipeline de décision Z2/D2 (découverte → allocation)
`validation.py`	Vérification multi-niveaux L1–L3 et métriques de qualité

8 Métriques de réussite du projet

Le projet doit produire un rapport complet pour chaque expérience menée.

Qualité	Perplexité, KL divergence, logit agreement, token agreement, erreur de reconstruction RMS/max
Mémoire	Taille totale, working set, RSS, VRAM, KV cache, résidence des experts
I/O	Octets/token (froids/chauds), cache hit rate, débit stockage, temps de chargement, décompression
Calcul	Temps kernel, temps GEMM, temps de déquantification, % de fallback CPU, occupation GPU/HTP
Performance	Prefill tok/s, decode tok/s, latence P50/P95, variance, tok/s/W si mesure énergétique
Prédiction	Écart prédiction/réalité, intervalle de confiance, erreur par backend et par format

9 Ordre de développement recommandé

P0	Préservation des acquis V3/V5	Ne rien casser de l'existant déjà validé
P0	Validation GTX 1080	Conserver la référence de qualité documentée
P1	Sensibilité et précision mixte	Améliorer le ratio qualité/taille
P1	Cost model	Mesurer et modéliser les vrais coûts matériels
P1	Working set et I/O	Optimiser les modèles dépassant la RAM
P2	Profilage MoE dynamique	Expert ID, traces de routage réel
P2	Cache expert adaptatif	Réduire drastiquement les lectures SSD
P2	Quantification par expert	Optimiser finement les poids MoE
P2	Compression physique	Réduire les transferts sans perte de qualité
P3	Z2/D2 adaptatif	Automatiser les décisions d'allocation
P3	Backend Qualcomm	Valider les kernels et les placements réels
P3	Optimisation MTP	Augmenter le débit de génération

10 Principes à ne pas perdre

Les treize règles d'or ci-dessous résument l'expérience accumulée sur les campagnes de mesure GTX 1080, GTX 1080 Ti et Qualcomm :

1. **Ne jamais confondre taille du checkpoint et working set.**
2. **Ne jamais confondre format compact et kernel rapide.**
3. **Ne jamais confondre compatibilité backend et accélération réelle.**
4. **Ne jamais considérer la perplexité comme l'unique mesure de qualité.**
5. **Ne jamais imposer une précision uniforme.**
6. **Ne jamais quantifier tous les experts de la même manière sans mesurer leur sensibilité.**
7. **Ne jamais considérer le cache LRU comme optimal par défaut.**
8. **Toujours séparer les octets demandés, transférés et réellement lus.**
9. **Toujours mesurer le coût de déquantification.**
10. **Toujours confronter les prédictions aux mesures matérielles.**
11. **Préserver les résultats GTX 1080 / 1080 Ti comme références expérimentales.**
12. **Préserver les travaux Qualcomm HTP / Adreno comme axe matériel à part entière.**
13. **Distinguer strictement les fonctionnalités implémentées, validées et encore expérimentales.**

11 Conclusion

Le projet **n'est pas simplement un quantificateur**. C'est une **architecture d'optimisation complète** qui associe :

- Analyse structurelle et numérique ;
- Quantification adaptative de précision mixte ;
- Profilage matériel multi-accélérateurs ;
- Gestion fine de mémoire et working-set contraint ;
- Optimisation des transferts et profilage MoE ;
- Prédiction de performance calibrée ;
- Planification multiobjectif sous contraintes ;
- Validation expérimentale rigoureuse à trois niveaux.

Les travaux sur GTX 1080 ont déjà démontré l'intérêt concret de l'allocation non uniforme de précision. Les travaux sur GTX 1080 Ti ont exploré les limites des représentations extrêmement compactes. Les travaux sur Qualcomm ont prouvé l'importance décisive de mesurer les placements et les fallbacks réels.

Profiler V3/V5 apporte la base de mesure et de prédiction. **Quantification Algo** apporte la recherche de représentations numériques optimales. **Z2/D2 Planner** apporte la sélection et l'orchestration des stratégies. La finalité est de **trouver automatiquement la meilleure représentation et la meilleure stratégie d'exécution pour un modèle, une qualité cible, un budget mémoire et un matériel donnés**.

Kimi K3 constitue le prochain terrain d'évaluation de cette architecture — il vient la prolonger, et non remplacer les acquis précédents.

12 Annexes

12.1 Structure recommandée du dépôt logiciel

```
quant_project/
├── doc/                        # cette documentation (source Typst + PDF)
├── figures/                    # diagrammes sources (Mermaid) et rendus PNG
├── src/adaptive_quant/         # bibliothèque Python
│   ├── capability_db.py        # profils matériels cibles
│   ├── quant_formats.py        # registre des formats numériques
│   ├── profile_model.py        # cartographie structurelle
│   ├── profiler.py             # profils mémoire & budgets
│   ├── predict_from_hf*.py     # prédiction de performances
│   ├── parse_hexagon.py        # traces Qualcomm HTP
│   ├── sensitivity_engine.py   # sensibilité par tenseur
│   ├── quantization_engine.py  # candidats & reconstruction
│   ├── mixed_precision.py      # allocation optimale de précision
│   ├── expert_profiler.py      # profilage dynamique MoE
│   ├── expert_cache.py         # simulateur de cache d'experts
│   ├── cost_model.py           # modèle de coûts T_token
│   ├── planner.py              # pipeline de décision Z2/D2
│   ├── validation.py           # vérification L1–L3
│   └── cli.py                  # interface en ligne de commande
├── tests/                      # suite de tests automatisés
└── examples/                   # scripts de démonstration (Kimi K3, etc.)
```

12.2 Glossaire des abréviations

AWQ	Activation-aware Weight Quantization
bpw	Bits per weight (bits effectifs par poids)
GGUF	Format de sérialisation de modèles (successeur de GGML)
GPTQ	Generalized Post-Training Quantization for Generative Pre-trained Transformers
HTP	Hexagon Tensor Processor (NPU Qualcomm)
KL	Divergence de Kullback-Leibler
MLA	Multi-head Latent Attention
MoE	Mixture of Experts
MTP	Multi-Token Prediction
QAIRT	Qualcomm AI Runtime
RMS	Root Mean Square (erreur quadratique moyenne)
SNR	Signal-to-Noise Ratio (rapport signal sur bruit)
VTCM	Vector Tightly-Coupled Memory (mémoire locale HTP)